

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE:CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

Coding Challenge

Code of Conduct:

I Atchaya Chandran R(192521394) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Atchay

Annexure A

Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate Secure Email Transmission using S/MIME, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement PGP-based file encryption and decryption, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate IPSec Tunnel Mode, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
4. Write a Python script to implement SSL/TLS handshake simulation, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform Email Phishing Detection, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Question 1: S/MIME Simulation

Python Implementation

Python

```
from cryptography import x509
```

```
from cryptography.hazmat.primitives import hashes, serialization
```

```
from cryptography.hazmat.primitives.asymmetric import padding, rsa
```

```

from cryptography.hazmat.primitives.ciphers.aead import AESGCM

import datetime

import os


def generate_cert(name):
    """Generates an RSA Key Pair and a Self-Signed Certificate."""
    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    subject = issuer = x509.Name(
        [x509.NameAttribute(x509.NameOID.COMMON_NAME, name)]
    )
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(issuer)
        .public_key(key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(
            datetime.datetime.now(datetime.timezone.utc)
            - datetime.timedelta(days=1)
        )
        .not_valid_after(
            datetime.datetime.now(datetime.timezone.utc)
            + datetime.timedelta(days=365)
        )
        .sign(key, hashes.SHA256())
    )

```

)

return key, cert

```
def smime_encrypt_and_sign(
```

```
    sender_key, sender_cert, receiver_cert, plain_text
```

```
):
```

```
    """1.
```

Sign message using Sender's Private Key 2. Encrypt message + signature

using Receiver's Public Key / Hybrid AES-GCM

```
    """
```

```
    message_bytes = plain_text.encode("utf-8")
```

```
# Step 1: Generate Digital Signature (Authentication & Integrity)
```

```
signature = sender_key.sign(
```

```
    message_bytes, padding.PKCS1v15(), hashes.SHA256()
```

```
)
```

```
# Combine message and signature
```

```
payload = message_bytes + b"::SIGNATURE::" + signature
```

```
# Step 2: Hybrid Encryption (Confidentiality)
```

```
session_key = AESGCM.generate_key(bit_length=256)
```

```
aesgcm = AESGCM(session_key)
```

```
nonce = os.urandom(12)
```

```
encrypted_payload = aesgcm.encrypt(nonce, payload, None)
```

```
# Encrypt session key with Receiver's RSA Public Key
encrypted_session_key = receiver_cert.public_key().encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    ),
)
```

```
return {
    "enc_session_key": encrypted_session_key,
    "nonce": nonce,
    "ciphertext": encrypted_payload,
}
```

```
def smime_decrypt_and_verify(
    receiver_key, sender_cert, encrypted_data
):
    """1.
```

Decrypt Session Key and Ciphertext 2. Extract Message & Verify Signature

"""

```
# Step 1: Decrypt Session Key
session_key = receiver_key.decrypt(
```

```
encrypted_data["enc_session_key"],
padding.OAEP(
    mgf=padding.MGF1(algorithm=hashes.SHA256()),
    algorithm=hashes.SHA256(),
    label=None,
),
)
```

Step 2: Decrypt Payload

```
aesgcm = AESGCM(session_key)
payload = aesgcm.decrypt(
    encrypted_data["nonce"], encrypted_data["ciphertext"], None
)
```

```
message_bytes, signature = payload.split(b":::SIGNATURE:::")
```

Step 3: Verify Signature

```
try:
    sender_cert.public_key().verify(
        signature, message_bytes, padding.PKCS1v15(), hashes.SHA256()
    )
    return message_bytes.decode("utf-8"), True
except Exception:
    return message_bytes.decode("utf-8"), False
```

--- Test Execution ---

```

sender_key, sender_cert = generate_cert("Alice (Sender)")
receiver_key, receiver_cert = generate_cert("Bob (Receiver)")

msg = "Confidential Payroll Data: $10,000"
encrypted_pkg = smime_encrypt_and_sign(
    sender_key, sender_cert, receiver_cert, msg
)
decrypted_msg, is_valid = smime_decrypt_and_verify(
    receiver_key, sender_cert, encrypted_pkg
)

print(
    f'Decrypted Message: {decrypted_msg}\nSignature Verified:'
    f' {is_valid}"
)

```

Analysis: Confidentiality and Authentication in S/MIME

- Confidentiality: Handled via symmetric encryption (AES-256-GCM) for the payload combined with asymmetric encryption (RSA-OAEP) for key transport. Only the recipient with the corresponding private key can decrypt the session key.
- Authentication & Non-repudiation: Ensured using digital signatures generated via the sender's RSA private key. The receiver uses the sender's X.509 certificate to verify that the message originated from the genuine sender and was not altered in transit.

Question 2: PGP-Based File Encryption & Decryption

Python Implementation

Python

```

from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa

```



```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os
```

```
class PGPNode:
```

```
    def __init__(self, name):
        self.name = name

        self.private_key = rsa.generate_private_key(
            public_exponent=65537, key_size=2048
        )

        self.public_key = self.private_key.public_key()

        self.key_ring = {} # Public Key Distribution/Store
```

```
    def add_public_key(self, user_name, pub_key):
        self.key_ring[user_name] = pub_key
```

```
    def encrypt_file(self, recipient_name, input_file, output_file):
        with open(input_file, "rb") as f:
            data = f.read()
```

```
        # Generate Signature
```

```
        signature = self.private_key.sign(
            data,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH,
```

```

    ),
    hashes.SHA256(),
)

# Encrypt Payload (Data + Signature) using AES
session_key = AESGCM.generate_key(bit_length=256)
aesgcm = AESGCM(session_key)
nonce = os.urandom(12)
payload = len(signature).to_bytes(4, "big") + signature + data
ciphertext = aesgcm.encrypt(nonce, payload, None)

# Encrypt Session Key with Recipient's Public Key
recipient_pub_key = self.key_ring[recipient_name]
enc_session_key = recipient_pub_key.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    ),
)

with open(output_file, "wb") as f:
    f.write(
        enc_session_key
        + nonce

```

```
        + len(enc_session_key).to_bytes(4, "big")
        + ciphertext
    )
```

```
def decrypt_file(self, sender_name, input_file, output_file):
```

```
    with open(input_file, "rb") as f:
```

```
        raw_data = f.read()
```

```
    # Parse binary structure
```

```
    enc_key_len = 256 # 2048-bit RSA = 256 bytes
```

```
    enc_session_key = raw_data[:enc_key_len]
```

```
    nonce = raw_data[enc_key_len : enc_key_len + 12]
```

```
    ciphertext = raw_data[enc_key_len + 16 :]
```

```
    # Decrypt session key
```

```
    session_key = self.private_key.decrypt(
```

```
        enc_session_key,
```

```
        padding.OAEP(
```

```
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
```

```
            algorithm=hashes.SHA256(),
```

```
            label=None,
```

```
        ),
```

```
    )
```

```
    # Decrypt payload
```

```
    aesgcm = AESGCM(session_key)
```

```
payload = aesgcm.decrypt(nonce, ciphertext, None)
```

```
sig_len = int.from_bytes(payload[:4], "big")
```

```
signature = payload[4 : 4 + sig_len]
```

```
data = payload[4 + sig_len :]
```

```
# Verify signature
```

```
sender_pub_key = self.key_ring[sender_name]
```

```
sender_pub_key.verify(
```

```
    signature,
```

```
    data,
```

```
    padding.PSS(
```

```
        mgf=padding.MGF1(hashes.SHA256()),
```

```
        salt_length=padding.PSS.MAX_LENGTH,
```

```
    ),
```

```
    hashes.SHA256(),
```

```
)
```

```
with open(output_file, "wb") as f:
```

```
    f.write(data)
```

```
return True
```

```
# --- Test Execution ---
```

```
alice = PGPNode("Alice")
```

```
bob = PGPNode("Bob")
```

```
alice.add_public_key("Bob", bob.public_key)
```

```
bob.add_public_key("Alice", alice.public_key)
```

```
with open("source.txt", "w") as f:
```

```
    f.write("PGP Confidential File Data")
```

```
alice.encrypt_file("Bob", "source.txt", "encrypted.pgp")
```

```
bob.decrypt_file("Alice", "encrypted.pgp", "decrypted.txt")
```

```
print("File successfully decrypted and signature verified.")
```

Evaluation: Security vs. Efficiency in PGP

- Security: Combines asymmetric cryptography (RSA) for key management and signature validation with symmetric encryption (AES-256) for data secrecy. This ensures strong integrity, non-repudiation, and confidentiality.
- Efficiency: Direct asymmetric encryption on large files is computationally expensive and slow. PGP achieves efficiency by encrypting the bulk data using fast symmetric algorithms (AES) and encrypting only the small session key with RSA.

Question 3: IPSec Tunnel Mode Simulation

Python Implementation

Python

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
```

```
import os
```

```
import struct
```

```
class IPSecTunnelMode:
```

```
    def __init__(self, key):
```

```
        self.key = key # Shared Pre-Shared Key (PSK) for ESP
```

```

def encapsulate(self, inner_ip_packet, outer_src, outer_dst):
    """Encapsulates and encrypts an ENTIRE IP packet inside a new IP Header
    (Tunnel Mode)."""
    # Symmetric AES-CBC Encryption
    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    encryptor = cipher.encryptor()

    # PKCS7 Padding
    pad_len = 16 - (len(inner_ip_packet) % 16)
    padded_payload = inner_ip_packet + bytes([pad_len] * pad_len)

    encrypted_esp_payload = encryptor.update(padded_payload) +
encryptor.finalize()

    # Constructing Simulated Outer IP Header + ESP Header
    esp_header = os.urandom(8) # SPI (4 bytes) + Sequence (4 bytes)
    outer_ip_header = f"OUTER_HDR|SRC:{outer_src}|DST:
{outer_dst}|".encode(
    "utf-8"
)

    tunneled_packet = outer_ip_header + esp_header + iv +
encrypted_esp_payload
    return tunneled_packet

def decapsulate(self, tunneled_packet):
    """Extracts outer header, decrypts payload, and retrieves the ORIGINAL

```

```
IP packet."""
```

```
parts = tunneled_packet.split(b"|", 3)
```

```
esp_data = parts[3]
```

```
esp_header = esp_data[:8]
```

```
iv = esp_data[8:24]
```

```
encrypted_payload = esp_data[24:]
```

```
cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
```

```
decryptor = cipher.decryptor()
```

```
padded_payload = decryptor.update(encrypted_payload) + decryptor.finalize()
```

```
# Remove PKCS7 Padding
```

```
pad_len = padded_payload[-1]
```

```
original_ip_packet = padded_payload[:-pad_len]
```

```
return original_ip_packet
```

```
# --- Test Execution ---
```

```
psk = os.urandom(32)
```

```
ipsec = IPSecTunnelMode(psk)
```

```
original_inner_packet = (
```

```
    b"INNER_HEADER|SRC:10.0.0.5|DST:10.0.0.9|PAYLOAD:Sensitive Data"
```

```
)
```

```
encapsulated = ipsec.encapsulate(
```

```

original_inner_packet, "192.168.1.1", "192.168.1.2"
)

decapsulated = ipsec.decapsulate(encapsulated)

print(f'Original Inner Packet: {original_inner_packet.decode()}')
print(f'Decapsulated Packet: {decapsulated.decode()}')

```

Analysis: Tunnel Mode vs. Transport Mode

Feature	Tunnel Mode	Transport Mode
Encapsulation	Encrypts the entire original IP packet (Header + Payload) and appends a new outer IP header.	Encrypts only the IP Payload (e.g., TCP/UDP segment); original IP header remains visible.
Use Case	Firewall-to-Firewall / Gateway-to-Gateway (VPNs).	End-to-End Host-to-Host communication.
Security	Hides inner IP topology and true source/destination end-hosts from intermediate networks.	Exposes source and destination IP addresses in transit.

Question 4: SSL/TLS Handshake Simulation

Python Implementation

Python

```

from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

```



```
class SSLTLSSimulator:
```

```
    def __init__(self):
```

```
        # Server Key and Certificate Setup
```

```
        self.server_private_key = rsa.generate_private_key(
```

```
            public_exponent=65537, key_size=2048
```

```
        )
```

```
        self.server_public_key = self.server_private_key.public_key()
```

```
    def simulate_handshake(self):
```

```
        # 1. ClientHello
```

```
        client_random = os.urandom(32)
```

```
        # 2. ServerHello & Certificate Exchange
```

```
        server_random = os.urandom(32)
```

```
        server_pub_key = self.server_public_key # "Certificate"
```

```
        # 3. Client Key Exchange
```

```
        pre_master_secret = os.urandom(48)
```

```
        encrypted_pre_master = server_pub_key.encrypt(
```

```
            pre_master_secret,
```

```
            padding.OAEP(
```

```
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
```

```
                algorithm=hashes.SHA256(),
```

```
                label=None,
```

```
            ),
```

)

4. Server Decrypts Pre-Master Secret

```
decrypted_pre_master = self.server_private_key.decrypt(
    encrypted_pre_master,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    ),
)
```

Both derive Key Derivation Function (KDF) Session Key

```
hkdf = hashes.Hash(hashes.SHA256())
hkdf.update(client_random + server_random + decrypted_pre_master)
session_key = hkdf.finalize()

return session_key
```

```
def send_secure_data(self, session_key, data):
```

```
    aesgcm = AESGCM(session_key)
    nonce = os.urandom(12)
    return nonce + aesgcm.encrypt(nonce, data.encode("utf-8"), None)
```

```
def receive_secure_data(self, session_key, encrypted_msg):
```

```
    nonce = encrypted_msg[:12]
    ciphertext = encrypted_msg[12:]
```

```

aesgcm = AESGCM(session_key)

return aesgcm.decrypt(nonce, ciphertext, None).decode("utf-8")

# --- Test Execution ---

tls = SSLTLSSimulator()

session_key = tls.simulate_handshake()

cipher_text = tls.send_secure_data(session_key, "GET /index.html HTTP/1.1")

plain_text = tls.receive_secure_data(session_key, cipher_text)

print(f'Derived Session Key (Hex): {session_key.hex()[:32]}...')

print(f'Decrypted Web Payload:    {plain_text}')

```

Evaluation: How Handshake Ensures Secure Web Communication

1. Authentication: The client verifies the server's identity through asymmetric key certificates, preventing Man-in-the-Middle (MitM) impersonation.
2. Key Exchange & Secrecy: The Pre-Master Secret is securely shared using asymmetric encryption (or Diffie-Hellman), guaranteeing that eavesdroppers cannot sniff the secret.
3. Symmetric Encryption Initialization: Once established, communication switches to fast symmetric algorithms (AES-GCM) with the derived session key, providing efficient privacy and data integrity.

Question 5: Email Phishing Detection Tool

Python Implementation

Python

```

import re

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics import classification_report

from sklearn.naive_bayes import MultinomialNB

```

```
# Dataset: Sample Email Corpus
```

```
emails = [
```

```
    "URGENT: Your account access is suspended! Verify password at  
    http://bit.ly/fake-login",
```

```
    "Dear Customer, update your credit card details immediately to avoid fees.",
```

```
    "Hey team, here are the meeting notes for our project sync up tomorrow.",
```

```
    "Please find attached the monthly invoice report for your review.",
```

```
    "SECURITY WARNING: Login detected from unknown device. Click  
    http://login-verify-bank.com",
```

```
    "Are we still on for lunch at 12:30 PM today?",
```

```
]
```

```
labels = [1, 1, 0, 0, 1, 0] # 1 = Phishing, 0 = Legitimate
```

```
# --- Feature Extraction Pipeline ---
```

```
def extract_heuristics(email_str):
```

```
    """Extracts custom domain/URL rule heuristics."""
```

```
    url_count = len(
```

```
        re.findall(
```

```
            r"http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[*\\(\)])|(?:%[0-9a-fA-F][0-9a-fA-  
F]))+",
```

```
            email_str,
```

```
        )
```

```
    )
```

```
    ip_url = 1 if re.search(r"http://[0-9]{1,3}\.[0-9]{1,3}", email_str) else 0
```

```
    urgent_keywords = (
```

```
        1
```

```

    if re.search(
        r"(urgent|suspended|verify|password|immediately|action required)",
        email_str,
        re.I,
    )
    else 0
)

return [url_count, ip_url, urgent_keywords]

```

NLP Machine Learning Feature Extraction using TF-IDF

```
vectorizer = TfidfVectorizer()
```

```
X_tfidf = vectorizer.fit_transform(emails)
```

Model Training

```
model = MultinomialNB()
```

```
model.fit(X_tfidf, labels)
```

Test Evaluation

```
test_email = [
    "URGENT: Click here http://suspicious-link.com to restore your bank"
    " account"
]
```

```
X_test = vectorizer.transform(test_email)
```

```
prediction = model.predict(X_test)
```

```
print(f"Sample Input Email: {test_email[0]}")
```

```
print(
```

```
    f'Classification Result: {'MALICIOUS / PHISHING' if prediction[0] == 1 else  
    'LEGITIMATE'}"
```

```
)
```

Effectiveness Analysis

- **NLP & Text Mining:** TF-IDF combined with Naive Bayes / Machine Learning provides high precision in catching social engineering phrases (e.g., "urgent", "suspended", "verify").
- **Structural Heuristics:** Identifying IP addresses in URLs, link shorteners, mismatched sender headers, and unexpected domain signatures complements textual analysis.
- **Limitations & Countermeasures:** Modern phishing attacks (zero-day attacks, obfuscated links, homograph attacks) can bypass simple keyword filters. Robust production pipelines integrate deep learning (BERT/Transformers) alongside SPF/DKIM/DMARC email protocol verification.